# AI – TECH

SKRYPT DO LABORATORIUM

## Przetwarzanie Multimediów w Systemach Decyzyjnych

LABORATORIUM

## Detekcja obiektów

**Sebastian Cygert**

Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego
Program Operacyjny Polska Cyfrowa na lata 2014-2020,
Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2 "Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej"
Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

# 1. Opis ćwiczenia

**Wymagania wstępne:**

**Cele ćwiczenia:**

**Spodziewane efekty kształcenia - umiejętności i kompetencje:**

**Metody dydaktyczne:**

**Zasady oceniania/warunek zaliczenia ćwiczenia**

**Wykaz literatury podstawowej do ćwiczenia:**

# 2. Przebieg ćwiczenia

# 3. Wprowadzenie do ćwiczenia

# 4. Forma i zawartość sprawozdania

# Dodatki

# 1. Zagadnienia wstępne

## 1.1.Wymagania w odniesieniu do studenta

W ramach przygotowania do zajęć, student powinien powtórzyć lub zapoznać się z podstawowymi zagadnieniami z działaniem sieci splotowych, analizą obrazu i detekcją obiektów. Będzie to możliwe poprzez:

- powtórzenie wiedzy nabytej w czasie wykładów,
- zapoznanie się (przed ćwiczeniem) z instrukcją do ćwiczenia ilustrującą podstawowe zagadnienia z zakresu tematyki ćwiczenia.

Ponadto wymagana jest od studenta:

- podstawowa umiejętność programowania,
- podstawowa znajomość języka Python,
- podstawowa znajomość algorytmów uczenia maszynowego przedstawionych w trakcie trwania wykładów,
- umiejętność twórczego myślenia.

Wyżej wymienione wymagania bazują na wiedzy i umiejętności jakie studenci powinni zdobyć w czasie edukacji na I poziomie studiów oraz w trakcie trwania 1 semestru studiów II stopnia. W przeciwnym przypadku studenci powinni nadrobić samodzielnie stosowny materiał.

## 1.2. Wymagania w odniesieniu do stanowiska laboratoryjnego

Stanowisko laboratoryjne powinno być wyposażone w komputer z dostępem do sieci komputerowej oraz z następującymi zasobami:

- przykładowe programy (kody programów) podane w załączniku
- zainstalowany język Python w wersji przynajmniej 3.6

- Jupyter Notebook
- Biblioteki niezbędne do realizacji ćwiczenia (wymienione w tabeli poniżej),
- inne: przeglądarka WWW.

Ponadto istnieje możliwość realizacji ćwiczenia w zewnętrznym środowisku Colaboratory na platformie Google, gdzie wymagane jest od użytkownika posiadanie konta Google, natomiast nie ma potrzeby konfiguracji środowiska. Wykaz najważniejszych bibliotek wykorzystywanych w czasie ćwiczenia:

| Nazwa biblioteki | Wersja | Zastosowanie w ćwiczeniu | Odnośnik |
|---|---|---|---|
| detectron2 | 0.6 | biblioteka udostępniająca wysokopoziomowe API do detekcji obiektów | https://github.com/facebookresearch/detectron2 |
| torch | 1.10 | biblioteka obsługująca modele detekcji obiektów | https://github.com/pytorch/pytorch |
| fiftyone | 0.14.1 | współpraca ze zbiorem danych COCO | https://github.com/voxel51/fiftyone |
| youtube-dl | v2021.06.06 | pobieranie nagrań z serwisu youtube | https://github.com/ytdl-org/youtube-dl |
| ffmpeg | 4.4 | wycinanie klatek z nagrań video | https://www.ffmpeg.org/ |

## 1.3.Cele ćwiczenia

Celem ćwiczenia jest praktyczna demonstracja wiedzy teoretycznej, przedstawionej w ramach wykładów. Realizując ćwiczenie laboratoryjne studenci zdobędą szereg nowych umiejętności w zakresie wykorzystania modeli detekcji obiektów z wykorzystaniem splotowych sieci neuronowych oraz

ewaluacji ich dokładności. Szczególny nacisk w ćwiczeniu kładziony jest na zrozumienie zależności między dokładnością modeli a wymaganiami obliczeniowy. Dodatkowo modele zostaną przetestowane w warunkach odbiegających od treningowych, co ma szczególne znaczenie praktyczne.

## 1.4. Spodziewane efekty kształcenia - umiejętności i kompetencje:

Studenci w ramach zajęć nabędą wiedzę dotyczącą między innymi następujących zadań:

- wyboru odpowiednich modeli do detekcji obiektów ze względu na wymaganą dokładność czy ograniczenia sprzętowe,
- systematycznej analizy dokładności model detekcji obiektów,
- zrozumienie ograniczeń współczesnych modeli rozpoznawania obrazu.

## 1.5.Metody dydaktyczne

Student w pierwszej kolejności realizuje zadania przykładowe podczas których przygotowuje odpowiednie środowisko programistyczne, poprzez które testuje poszczególne elementy pracy z danymi do których można wykorzystać metodę omawiane wcześniej algorytmy nadzorowanego uczenia maszynowego. Następnie, na podstawie posiadanej wiedzy i zdobytej umiejętności z zadań demonstracyjnych, realizuje przydzielone zadania, których poziom trudności wzrasta wraz z kolejnymi przykładami. Dodatkowo pośród poleceń pojawiają się pytania dotyczące zagadnień teoretycznych, sprawdzające przygotowanie studentów do zajęć. Wszystkie kody źródłowe programów wpisuje w dokumencie elektronicznym. Wyniki zapisywane są w osobnym pliku ze sprawozdaniem (format dowolny, np. docx).

Materiały wprowadzające i pomocnicze:

- Python – dokumentacja języka https://docs.python.org/pl/3/

- Jupiter notebook – narzędzie ułatwiające implementację kodu, uruchamianie oraz testowanie: https://realpython.com/jupyter-notebook-introduction/
- Google Colab – narzędzie umożliwiające pracę z kodem w notebooku Jupiter oraz jego uruchamianie w chmurze (konieczność posiadania konta Google): https://colab.research.google.com/github/cs231n/cs231n.github.io/blob/master/python-colab.ipynb

### 1.5.Zasady oceniania

Ocenie podlegać będzie realizacja poszczególnych zadań w ramach danego ćwiczenia laboratoryjnego. Poszczególnym zadaniom przydzielono liczbę punktów jakie można zdobyć za ich wykonania. Maksymalna liczba punktów wynosi 10.

**Wykaz literatury podstawowej do ćwiczenia:**

**Wykaz literatury podstawowej:**

| | |
|---|---|
| 1. | Skrypt z materiałami do przedmiotu „Uczenie Maszynowe” |
| 2. | Materiały do przedmiotu opracowane w formie edukacji na odległość, dostęp: https://enauczanie.pg.edu.pl/moodle/course/view.php?id=12525 |
| 3. | Sebastian Raschka, Python. Uczenie maszynowe, Helion 2017 |

**Wykaz literatury uzupełniającej:**

| | |
|---|---|
| 1. | Dokumentacja platformy Colaboratory: https://colab.research.google.com/notebooks/intro.ipynb#scrollTo=-gE-Ez1qtyIA |
| 2. | Francois Chollet, Deep Learning. Praca z językiem Python i biblioteką Keras, Helion 2019 |

## 2. Przebieg ćwiczenia

| L.p. | Zadanie |
|---|---|
| 1. | Zapoznanie się z instrukcją laboratoryjną (przed ćwiczeniem) |
| 2. | Wprowadzenie kierownika ćwiczenia. Zapoznanie się uczestników ze stosowanymi programami, przegląd dokumentacji (5 min.) |

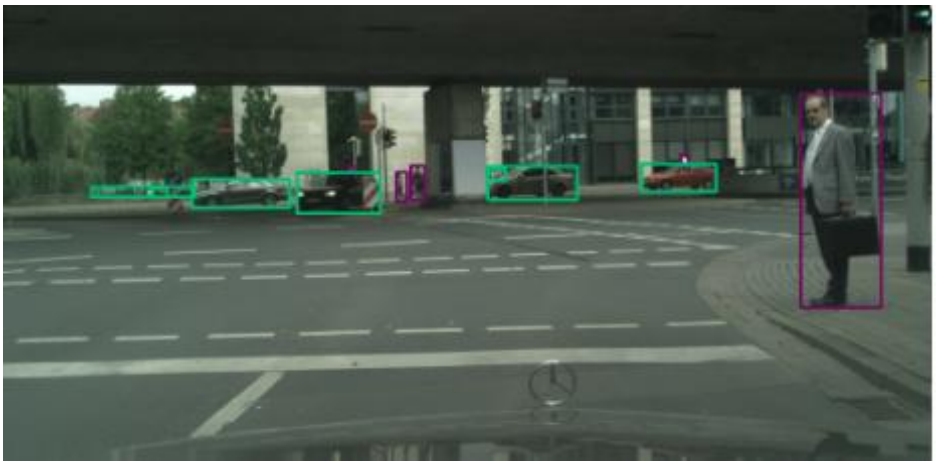
| | |
|---|---|
| 3. | Wprowadzenie do detekcji obietków (10 min.); |
| 4. | Ewaluacja na zbiorze COCO (10 min.); |
| 5. | Porównanie skuteczności działania różnych modeli (25 min.) |
| 6. | Zbadanie odporności modeli detekcji obiektów na szum gaussowski (20 min.) |
| 7. | Zbadanie skuteczności działania modeli na nagraniach video (15 min.) |
| 8. | Przesłanie/pokazanie zbioru dokumentów/programów do oceny (5 min.). |

Przed przystąpieniem do ćwiczenia należy utworzyć własny, unikalny katalog w lokalizacji wskazanej przez prowadzącego, a następnie należy tam umieścić własne implementacje kodu programów dokonujących rozwiązanie zadania oraz sprawozdanie.

## 3. Wprowadzenie do ćwiczenia

Na zajęciach zajmiemy się problemem detekcji obiektów. W zadaniu tym celem algorytmu jest wskazanie gdzie na **obrazie wejściowym** znajdują się obiekty należące do klas zainteresowania. Lokalizacja odbywa się zazwyczaj poprzez podanie lokalizacji (w pikselach) prostokąta opisującego wybrane obiekty. Typowe zastosowania to:

- monitoring (ruch drogowy, monitoring miejski, monitoring ludzi na lotnisku),
- wykrywanie potencjalnie niebezpiecznych zdarzeń,
- robotyka (detekcja obiektów, przeszkód).



Rys. 1. Przykład detekcji obiektów ze zbioru Cityscapes [1].

Modele detekcji obiektów najczęściej trenowane w sposób nadzorowany (ang. *supervised learning*). Typowe wyzwania towarzyszące detekcji obiektów to:

- okluzje - przesłanianie instancji obiektów przez inne obiekty (również należące do tej samej klasy),
- zmienne oświetlenie, w szczególności detekcja nocą,
- zmienne warunki atmosferyczne,
- występowanie rzadkich zdarzeń (które np. mogły nie występować w zbiorze treningowym).

Najpopularniejsze modele do detekcji obiektów oparte są o splotowe sieci neuronowe (ang. *Convolutional Neural Networks, CNNs*). W szczególności popularna jest architektura Faster R-CNN [2], która działa w dwóch etapach:

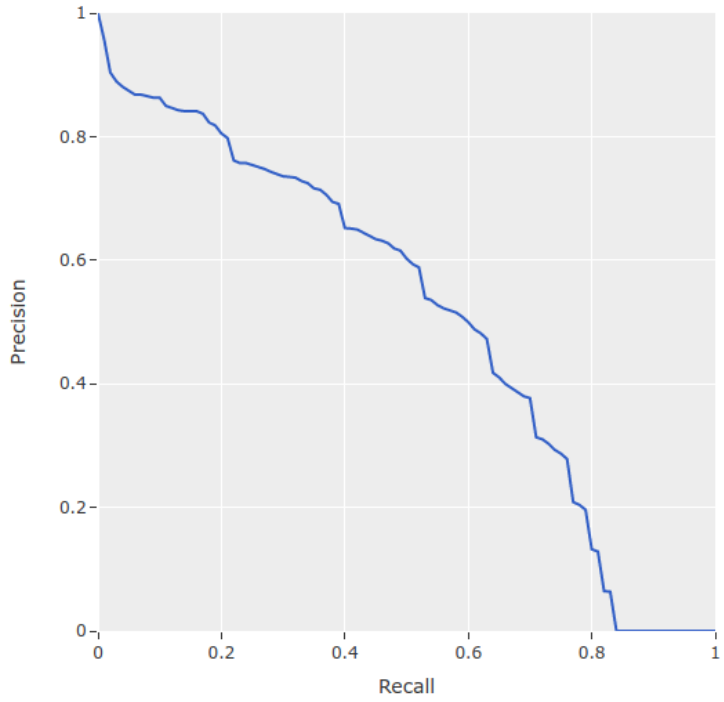
- ekstrakcja obszarów zainteresowania. Celem jest znalezienie obszarów, które mogą zawierać poszukiwane obiekty. Współczesne modele zwracają około 200-300 propozycji na obraz (liczba ta się może zmieniać w zależności od zastosowania)

- klasyfikacja. Każdy z obszarów zainteresowania jest klasyfikowany jako posiadający wybrane obiekty lub zawierający tylko tło.
-

Następnie model zwraca detekcje dla których sieć zwraca prawdopodobieństwo obiektu większe niż zadany próg (ang. threshold) - zazwyczaj jest to wartość w okolicach 0.5. W końcowym etapie usuwane są redundantne detekcje korzystając z algorytmu *Non-maximum Suppression (NMS).*

Współczesne modele wymagają ogromnej liczby danych aby zostać poprawnie wytrenowane. W celu zmniejszenia wymogów danych, często stosowane jest uczenie transferowe (ang. *transfer learning*). Oznacza to, że model jest wstępnie wytrenowywany na jednym z dużych, publicznie dostępnych zbiorów danych a następnie ostatnie warstwy sieci są dotrenowywane (ang. *finetuning*) już na docelowym zbiorze danych. W takiej często już kilkaset zdjęć z etykietami wystarczy aby skutecznie wytrenować model. Najpopularniejszym modelem do wstępnego treningu w detekcji obiektów jest zbiór MS-COCO [3] zawierający ponad 300 tys. zdjęć z adnotacjami dla 90 różnych klas.

**Ewaluacja.** Aby zmierzyć dokładność danego modelu na wybranym zbiorze danych należy każdą detekcję zwrócona przez modele zaakceptować jako poprawną (*True Positive*) lub odrzucić jako błędną (*False Positive*). Dodatkowo należy znaleźć wszystkie obiekty pominięte przed model (*False Negative*). Mając takie dane można przystąpić do stworzenia wykresu krzywej precyzja-odwołanie (ang. *precision recall curve*), dla różnych progów pewności modelu. Taką krzywą można wyznaczyć dla każdej z klas z osobna, a pole pod takim wykresem wyznacza metrykę średniej dokładności (ang. average precision, AP).




Rys. 2. Przykładowy wykres precyzji-odwołania.

Współczesne modele detekcji obiektów (czy rozpoznawania obrazu w ogólności) dobrze sobie radzą gdy warunki testowe są zbliżone do treningowych. Jednak gdy warunki te się znacząco różnią to dokładność modeli potrafi drastycznie spadać. Taki brak zdolności generalizacji współczesnych modeli do nowych warunków testowych jest jednym z największych wyzwań współczesnego uczenia maszynowego i pokazuje jedna ze słabości tych algorytmów, w porównaniu do tego jak działa np. ludzka percepcja.

Celem ćwiczenia jest:

- zaznajomienie się z dostępnymi bibliotekami i gotowymi modelami do detekcji obiektów,
- zbadanie zależności między dokładnością modeli a czasem inferencji,
- zapoznanie się z metrykami i sposobem ewaluacji modeli detekcji obiektów,
- zbadanie możliwości adaptacji gotowych modeli do nowych warunków testowych i ich wrażliwości na syntetyczny szum.

Szczegółowe opisy oraz poszczególne moduły kodów zostały umieszczone w skryptach jupyter notebook, które są nierozłącznym elementem instrukcji do tego ćwiczenia laboratoryjnego.

## 4. Forma i zawartość sprawozdania

Ocenie podlegać będzie realizacja poszczególnych zadań w ramach danego ćwiczenia laboratoryjnego. Maksymalna liczba punktów wynosi 10. Oprócz wykonanego kodu należy załączyć sprawozdanie zgodnie z instrukcją w pliku jupyter-notebook.X

### Dodatki:

Pliki instruktażowe oraz pliki z zdanymi niezbędne do wykonania ćwiczenia laboratoryjnego:

1) *lab_detekcja.ipynb*